Quantstamp

Polymarket - Perpetual Contract Updates

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Polymarket
Timeline	2026-07-06 through 2026-07-07
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Diff/Fork information	Last fix review commit `c996c61592a64bf104cae754ea5d553a02bbcba5` and last audit commit `7dd0575cd0da60881738bded7b6aa1a231ba2147`
Source Code	• Polymarket/perpetuals-contract #12ed980
Auditors	• Tim Sigl Auditing Engineer • Zeeshan Meghji Senior Auditing Engineer • Paul Clemson Auditing Engineer

Documentation quality	Medium
Test quality	High
Total Findings	0
High severity findings	0
Medium severity findings	0
Low severity findings	0
Undetermined severity findings	0
Informational findings	0

Summary of Findings

The exchange is a custody and settlement contract for an off-chain perpetuals exchange. Users deposit ERC-20 collateral, trade off-chain, and the operator periodically commits a Merkle root over `(account, token, balance)` leaves. Withdrawals are processed by a keeper and proven against the last committed root.

This audit covers an update to the contract previously audited (last fix review commit `c996c61592a64bf104cae754ea5d553a02bbcba5`). The update strips the protocol down to a deposit, state-root commit, and keeper-run withdrawal core. Key changes versus the prior audit:

- Removed the escape hatch, the on-chain `requestWithdrawal()` queue, the withdrawal cooldowns, and the recovery flow (`prepareResume()` / `clearQueue()` / `resume()`).
- Added a `keeper` role that runs off-chain withdrawals, operators now only commit roots, and the two roles are mutually exclusive.
- Re-keyed the withdrawal mapping from the state root to an epoch counter that increments on every commit.
- Changed the EIP-712 `Withdraw` type (`uint64 salt`, `uint64 ts`); signatures are valid for one hour.
- Made signature verification EIP-7702 aware (ECDSA first, ERC-1271 fallback for addresses with code).
- Deposits now credit the measured balance delta.

No findings were identified during this review. A few auditor suggestions are attached to further strengthen the codebase.

The Foundry suite passes with unit tests only (no fuzz, invariant, or integration tests). The in-scope contract has 93.10% branch and 100% function coverage.

Fix-Review Update 2026-07-07:
In order to deploy the contract upgrade with the minimum necessary changes, the client elected to acknowledge the auditor suggestions presented in the report.

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

> **Disclaimer**
>
> Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
    1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
    2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
    3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
    1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
    2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: `https://github.com/Polymarket/perpetuals-contract`

Included Paths: `src/v1`

Operational Considerations

Withdrawal accounting resets on every state root commit: the payout mapping `withdrawn[epoch][token][account]` is keyed by the `epoch` counter, which `commitStateRoot()` increments on every commit, so amounts already paid out are forgotten once a new root lands. The client chose epoch keying because the same state root can legitimately recur across commits, but as a consequence any committed root that does not deduct withdrawals settled in earlier epochs lets the affected accounts prove the stale balance and withdraw again, draining collateral backing other users. The contract enforces no cross-epoch protection itself. The client indicated that the off-chain engine is responsible for reflecting every `WithdrawalCompleted` settlement in the next committed root, and this high-trust operational assumption was considered throughout the audit.

Signed withdrawals are executable for one hour and cannot be revoked: `_verifyWithdrawal()` accepts an EIP-712 signature whose `ts` is up to 1 hour in the past, and there is no on-chain cancellation, so a keeper holding a signature can submit it through `processWithdrawal()` at any point in that window, sending funds to the signed `to` address and retaining the signed `fee`. Users should treat signing a withdrawal as final, and keeper operations are expected to submit signatures promptly and handle unsubmitted ones as sensitive material.

The root EOA key remains a live authorization path for EIP-7702 delegated accounts: `_isValidSignature()` attempts ECDSA recovery before any ERC-1271 check, so the account's original private key authorizes `processWithdrawal()` even when delegated code would reject the signature. This matches EIP-7702 root authority, but delegate-level policies such as session keys, spending limits, or key rotation cannot restrict or revoke root-key withdrawal signatures. Users of delegated accounts should treat custody of the EOA key as full withdrawal authority on the exchange.

Key Actors And Their Capabilities

Owner

- Manages the operator and keeper roles and supported assets, transfers ownership (two-step), and authorizes UUPS upgrades. A single address cannot hold both the operator and keeper roles.

Operator

- Commits Merkle state roots via `commitStateRoot()`.

Keeper

- Processes off-chain-signed withdrawals via `processWithdrawal()`, verifying the account's EIP-712 signature and a Merkle proof against the committed root.

Auditor Suggestions

S1 Change `timestamp` Parameter Naming in `processWithdrawal()` to Match `WITHDRAW_TYPEHASH` Signature

Acknowledged

File(s) affected: `src/v1/ExchangeV1.sol`

Description: The `processWithdrawal()` function takes a `uint64 timestamp` argument; however, the `WITHDRAW_TYPEHASH` expects this parameters to be `uint64 ts`. If offchain signers were to sign a struct where the field name was `timestamp`, it would compute a different typehash and cause the signature to fail.

Recommendation: Align the parameter names in the `processWithdrawal()` function to match those in the `WITHDRAW_TYPEHASH` to increase consistency and reduce the possibility of off-chain signer errors.

S2 Improve Validations

Acknowledged

File(s) affected: `src/v1/ExchangeV1.sol`

Description: The following functions could benefit from improved validations before making a duplicate storage write and emitting an unnecessary event:

- In `setOperator()` check that `operators[_operator] != _enabled` and `_operator != address(0)`
- In `setKeeper()` check that `keepers[_keeper] != _enabled` and `_keeper != address(0)`
- In `setAsset()` check that `supportedAssets[_token] != minAmount`

Recommendation: Consider adding the suggested validations.

S3 Simplify Codebase

Acknowledged

File(s) affected: `src/v1/ExchangeV1.sol`

Description: The functions `_remainingWithdrawable()` and `_settleWithdrawal()` both accept an `_epoch` argument, but all current callers to these internal functions pass the current `epoch` state variable. If historical-epoch accounting is not intended, these helpers could read `epoch` directly. This would slightly simplify the call sites and make the helpers' dependency on the current accounting window explicit.

Recommendation: Consider implementing the recommended simplification.

S4 Increase Signature Timestamp Error Clarity

Acknowledged

File(s) affected: `src/v1/ExchangeV1.sol`

Description: The `_verifyWithdrawal()` function validates the timestamp of the user's signature in the following check:

```
if (ts > block.timestamp || block.timestamp - ts > 1 hours) {
    revert SignatureExpired();
}
```

If the timestamp is greater than one hour old, or if the timestamp provided is in the future, the call will revert with the `SignatureExpired()` error despite a future timestamp not being "expired".

Recommendation: Consider one of two alternatives:

1. Separate the checks to have two errors, for example `SignatureExpired()` and `FutureSignaturesInvalid()`

2. Rename the `SignatureExpired()` error to something that encompasses both scenarios, such as `InvalidSignatureTimestamp()`

S5

Update Documentation to Match the Epoch-Keyed Withdrawal Mapping and Storage Layout

Acknowledged

File(s) affected: `README.md`, `claude.md`

Description: Project documentation still describes the previous implementation in two places:

1. README.md states the contract tracks `withdrawn[root][token][account]` and frames operator guarantee three around the "last committed root," while `StorageV1` keys the mapping as `withdrawn[epoch][token][account]` and `commitStateRoot()` increments `epoch` on every commit.
2. README.md describes multiple withdrawal channels feeding a FIFO queue. In the `ExchangeV1` implementation, withdrawals are processed directly through keeper-submitted `processWithdrawal()`; there is no FIFO withdrawal queue, `requestWithdrawal()`, fulfillment, or rejection flow.
3. claude.md describes V1 storage as slots 0-7 and V2 storage as slots 8-19, while the generated `.storage-layout` places V1 storage at slots 0-8 (`withdrawn` at slot 8) and V2 storage at slots 9-20.

Recommendation: Consider updating README.md and claude.md to describe the epoch-keyed mapping and the actual slot ranges.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/Polymarket/perpetuals-contract`

- `eae...e92 ./src/v1/ExchangeV1.sol`
- `240...122 ./src/v1/IExchangeV1.sol`
- `d88...5e6 ./src/v1/StorageV1.sol`

Test Suite Results

Test data output was obtained with `forge test`. All 2 test suites with 151 tests in total executed successfully. The client should consider adding targeted regression tests for each finding to verify that fixes are implemented correctly and remain effective in future updates.

```
Compiling 4 files with Solc 0.8.34
Solc 0.8.34 finished in 15.16s
Compiler run successful!

Ran 62 tests for test/ExchangeV1.t.sol:ExchangeV1Test
[PASS] test_acceptOwnership_revertsIfNoPending() (gas: 19985)
[PASS] test_acceptOwnership_revertsIfNotPendingOwner() (gas: 48130)
[PASS] test_cancelPendingTransfer() (gas: 38920)
[PASS] test_cancelPendingTransfer_revertsIfNoPending() (gas: 21300)
[PASS] test_cancelPendingTransfer_revertsIfNotOwner() (gas: 48098)
[PASS] test_commitStateRoot() (gas: 69015)
[PASS] test_commitStateRoot_revertsIfNotOperator() (gas: 18095)
[PASS] test_commitStateRoot_revertsOnZeroRoot() (gas: 18290)
[PASS] test_deposit() (gas: 62950)
[PASS] test_depositWithPermit() (gas: 70501)
[PASS] test_depositWithPermit_noPreApproval() (gas: 97859)
[PASS] test_depositWithPermit_propagatesPermitRevert() (gas: 58230)
[PASS] test_depositWithPermit_revertsBelowMinDeposit() (gas: 76458)
[PASS] test_depositWithPermit_revertsOnZeroAddress() (gas: 30928)
[PASS] test_depositWithPermit_revertsOnZeroAmount() (gas: 30462)
[PASS] test_depositWithPermit_skipsPermitWhenAllowanceSufficient() (gas: 63198)
[PASS] test_deposit_allowsExactMinDeposit() (gas: 74908)
[PASS] test_deposit_emitsEvent() (gas: 63594)
[PASS] test_deposit_revertsBelowMinDeposit() (gas: 69845)
[PASS] test_deposit_revertsOnUnsupportedToken() (gas: 806088)
[PASS] test_deposit_revertsOnZeroAddress() (gas: 19598)
[PASS] test_deposit_revertsOnZeroAmount() (gas: 20041)
[PASS] test_initialize_revertsOnDoubleInit() (gas: 19586)
[PASS] test_initialize_revertsOnZeroAddress() (gas: 1670763)
[PASS] test_mockERC20_decimals() (gas: 7664)
[PASS] test_processWithdrawal() (gas: 193213)
[PASS] test_processWithdrawal_eip7702_delegatedEOA_ecdsaAccepted() (gas: 195495)
[PASS] test_processWithdrawal_eip7702_ecdsaWinsOverRejectingDelegate() (gas: 366132)
[PASS] test_processWithdrawal_eip7702_fallsBackToERC1271() (gas: 370588)
[PASS] test_processWithdrawal_eip7702_revertsWhenNeitherPathValid() (gas: 320402)
[PASS] test_processWithdrawal_revertsIfNotKeeper() (gas: 130986)
[PASS] test_processWithdrawal_revertsIfOperatorNotKeeper() (gas: 131141)
[PASS] test_processWithdrawal_revertsOnExpiredSignature() (gas: 138200)
[PASS] test_processWithdrawal_revertsOnFeeTooHigh() (gas: 36908)
[PASS] test_processWithdrawal_revertsOnFutureTimestamp() (gas: 136408)
[PASS] test_processWithdrawal_revertsOnInsufficientBalanceAfterPartial() (gas: 203719)
[PASS] test_processWithdrawal_revertsOnInvalidERC1271Signature() (gas: 317605)
[PASS] test_processWithdrawal_revertsOnInvalidSignature() (gas: 142746)
[PASS] test_processWithdrawal_revertsOnNoStateRoot() (gas: 38110)
[PASS] test_processWithdrawal_revertsOnZeroAddressTo() (gas: 37521)
[PASS] test_processWithdrawal_revertsOnZeroAmount() (gas: 36619)
[PASS] test_processWithdrawal_supportsERC1271() (gas: 388659)
[PASS] test_processWithdrawal_withFee() (gas: 193241)
[PASS] test_setAsset() (gas: 822828)
[PASS] test_setAsset_revertsIfNotOwner() (gas: 21112)
[PASS] test_setAsset_revertsOnZeroAddress() (gas: 19205)
[PASS] test_setKeeper() (gas: 40868)
[PASS] test_setKeeper_disableDoesNotCheckOperator() (gas: 30372)
[PASS] test_setKeeper_revertsIfAlreadyOperator() (gas: 24484)
[PASS] test_setKeeper_revertsIfNotOwner() (gas: 22161)
[PASS] test_setOperator() (gas: 39131)
[PASS] test_setOperator_allowsRoleSwapAfterRevoke() (gas: 60649)
[PASS] test_setOperator_disableDoesNotCheckKeeper() (gas: 29599)
[PASS] test_setOperator_revertsIfAlreadyKeeper() (gas: 22768)
[PASS] test_setOperator_revertsIfNotOwner() (gas: 21765)
[PASS] test_transferOwnership_emitsEvents() (gas: 39425)
[PASS] test_transferOwnership_overwritesPendingOwner() (gas: 47159)
[PASS] test_transferOwnership_revertsIfNotOwner() (gas: 20429)
[PASS] test_transferOwnership_revertsOnZeroAddress() (gas: 19815)
[PASS] test_transferOwnership_twoStep() (gas: 42127)
[PASS] test_upgrade_authorizedByOwner() (gas: 1609504)
[PASS] test_upgrade_revertsIfNotOwner() (gas: 1605298)
Suite result: ok. 62 passed; 0 failed; 0 skipped; finished in 157.80ms (367.18ms CPU time)

Ran 89 tests for test/ExchangeV2.t.sol:ExchangeV2Test
```

```
[PASS] test_clearQueue_clearsInBatches() (gas: 408358)
[PASS] test_clearQueue_revertsIfNotOwner() (gas: 20539)
[PASS] test_clearQueue_revertsIfNotPrepared() (gas: 23147)
[PASS] test_commitStateRoot_revertsInEscapeHatch() (gas: 408238)
[PASS] test_commitStateRoot_revertsWithPendingRequests() (gas: 406978)
[PASS] test_commitStateRoot_succeedsAfterFulfill() (gas: 411133)
[PASS] test_depositWithPermit_revertsInEscapeHatch() (gas: 415540)
[PASS] test_escapeHatch_autoActivatesOnExpiredRequest() (gas: 408948)
[PASS] test_escapeHatch_blocksDeposit() (gas: 410872)
[PASS] test_escapeHatch_blocksOperatorActions() (gas: 416482)
[PASS] test_escapeHatch_blocksProcessWithdrawal() (gas: 428388)
[PASS] test_escapeHatch_notActiveIfFulfilledBeforeDeadline() (gas: 405672)
[PASS] test_fulfillWithdrawal() (gas: 408052)
[PASS] test_fulfillWithdrawal_resetsCooldown() (gas: 625048)
[PASS] test_fulfillWithdrawal_revertsIfAlreadyFulfilled() (gas: 408038)
[PASS] test_fulfillWithdrawal_revertsIfNotOperator() (gas: 410233)
[PASS] test_fulfillWithdrawal_revertsIfQueuedAmountNoLongerAvailable() (gas: 533164)
[PASS] test_fulfillWithdrawal_revertsOnCorruptedZeroRoot() (gas: 433324)
[PASS] test_fulfillWithdrawal_withFee() (gas: 402553)
[PASS] test_getPendingWithdrawals_empty() (gas: 16184)
[PASS] test_inEscapeHatch_returnsFalseInitially() (gas: 18262)
[PASS] test_prepareResume_resetsOnReactivation() (gas: 465648)
[PASS] test_prepareResume_revertsIfNotEscapeHatch() (gas: 25610)
[PASS] test_prepareResume_revertsIfNotOwner() (gas: 409321)
[PASS] test_prepareResume_setsFlagAndKeepsQueue() (gas: 468675)
[PASS] test_processWithdrawal_eip7702_delegatedEOA_ecdsaAccepted() (gas: 232579)
[PASS] test_processWithdrawal_eip7702_ecdsaWinsOverRejectingDelegate() (gas: 402891)
[PASS] test_processWithdrawal_eip7702_fallsBackToERC1271() (gas: 407523)
[PASS] test_processWithdrawal_eip7702_revertsWhenNeitherPathValid() (gas: 330478)
[PASS] test_processWithdrawal_revertsOnInvalidERC1271Signature() (gas: 327963)
[PASS] test_processWithdrawal_revertsOnReplayedSalt() (gas: 240593)
[PASS] test_processWithdrawal_supportsERC1271() (gas: 425241)
[PASS] test_rejectWithdrawal() (gas: 404697)
[PASS] test_rejectWithdrawal_cooldown24h() (gas: 659053)
[PASS] test_rejectWithdrawal_cooldownCapsAtMax() (gas: 380987)
[PASS] test_rejectWithdrawal_cooldownCapsOnHighCount() (gas: 377080)
[PASS] test_rejectWithdrawal_exponentialBackoff() (gas: 1058064)
[PASS] test_rejectWithdrawal_revertsIfAlreadyFulfilled() (gas: 406420)
[PASS] test_rejectWithdrawal_revertsIfNotOperator() (gas: 409557)
[PASS] test_requestWithdrawal() (gas: 412536)
[PASS] test_requestWithdrawal_revertsIfAlreadyPending() (gas: 410415)
[PASS] test_requestWithdrawal_revertsIfNoStateRoot() (gas: 74270)
[PASS] test_requestWithdrawal_revertsInEscapeHatch() (gas: 408842)
[PASS] test_requestWithdrawal_revertsOnFeeTooHigh() (gas: 28529)
[PASS] test_requestWithdrawal_revertsOnInsufficientBalance() (gas: 135364)
[PASS] test_requestWithdrawal_revertsOnInvalidProof() (gas: 132466)
[PASS] test_requestWithdrawal_revertsOnZeroAddressTo() (gas: 26679)
[PASS] test_requestWithdrawal_revertsOnZeroAmount() (gas: 26788)
[PASS] test_requestWithdrawal_revertsWhenCapReached() (gas: 442626)
[PASS] test_requestWithdrawal_withFee() (gas: 407823)
[PASS] test_resume_recoversAfterPrepare() (gas: 402547)
[PASS] test_resume_revertsDuringGracePeriod() (gas: 409094)
[PASS] test_resume_revertsIfNotOwner() (gas: 20093)
[PASS] test_resume_revertsIfNotPrepared() (gas: 436488)
[PASS] test_resume_revertsIfQueueNotCleared() (gas: 464164)
[PASS] test_resume_revertsOnZeroStateRoot() (gas: 410514)
[PASS] test_setResumeGracePeriod() (gas: 36067)
[PASS] test_setResumeGracePeriod_revertsIfAboveMax() (gas: 27059)
[PASS] test_setResumeGracePeriod_revertsIfBelowMin() (gas: 26354)
[PASS] test_setResumeGracePeriod_revertsIfNotOwner() (gas: 19175)
[PASS] test_setResumeGracePeriod_revertsInEscapeHatch() (gas: 413747)
[PASS] test_setWithdrawCooldowns() (gas: 35982)
[PASS] test_setWithdrawCooldowns_revertsIfBaseAboveMax() (gas: 20206)
[PASS] test_setWithdrawCooldowns_revertsIfBaseBelowMin() (gas: 20073)
[PASS] test_setWithdrawCooldowns_revertsIfBaseGtMax() (gas: 20359)
[PASS] test_setWithdrawCooldowns_revertsIfMaxAboveMax() (gas: 19347)
[PASS] test_setWithdrawCooldowns_revertsIfMaxBelowMin() (gas: 19192)
[PASS] test_setWithdrawCooldowns_revertsIfNotOwner() (gas: 19730)
[PASS] test_setWithdrawalTimeout() (gas: 33449)
[PASS] test_setWithdrawalTimeout_allowsIncreasingWithPendingRequests() (gas: 416014)
[PASS] test_setWithdrawalTimeout_revertsIfAboveMax() (gas: 19299)
[PASS] test_setWithdrawalTimeout_revertsIfBelowMin() (gas: 19320)
```

```
[PASS] test_setWithdrawalTimeout_revertsIfNotOwner() (gas: 20495)
[PASS] test_setWithdrawalTimeout_revertsIfShorteningWithPendingRequests() (gas: 411026)
[PASS] test_upgrade_fromV1ToV2_preservesStateAndEnablesV2() (gas: 5073692)
[PASS] test_upgrade_initializeV2_revertsOnSecondCall() (gas: 17294)
[PASS] test_upgrade_v2ToV2_authorizedByOwner() (gas: 2686901)
[PASS] test_upgrade_v2ToV2_revertsIfNotOwner() (gas: 2683083)
[PASS] test_withdraw_autoActivates() (gas: 439121)
[PASS] test_withdraw_capsToAvailableBalance() (gas: 450798)
[PASS] test_withdraw_negativeBalanceReverts() (gas: 453931)
[PASS] test_withdraw_revertsIfAlreadyClaimed() (gas: 443611)
[PASS] test_withdraw_revertsIfNoExpiredRequests() (gas: 126125)
[PASS] test_withdraw_revertsIfNoStateRoot() (gas: 20298)
[PASS] test_withdraw_revertsOnInvalidProof() (gas: 433748)
[PASS] test_withdraw_revertsOnZeroAvailableBalance() (gas: 423848)
[PASS] test_withdraw_revertsOnZeroBalance() (gas: 434731)
[PASS] test_withdraw_twoUsers() (gas: 491070)
[PASS] test_withdraw_withMerkleProof() (gas: 443157)
Suite result: ok. 89 passed; 0 failed; 0 skipped; finished in 157.79ms (280.33ms CPU time)

Ran 2 test suites in 218.35ms (315.59ms CPU time): 151 tests passed, 0 failed, 0 skipped (151 total
tests)
```

Code Coverage

The code coverage output was obtained by running `forge coverage --ir-minimum --optimizer-runs 300 --no-match-coverage "test/|script/|src/v2"`. The full test suite is executed; only the in-scope src/v1/ExchangeV1.sol is reported, and the out-of-scope V2 contract is excluded from the table.

The test suite has high branch coverage of 93.10%, meaning that most paths are well tested.

File	% Lines	% Statements	% Branches	% Funcs
src/v1/ExchangeV1.sol	83.06% (103/124)	83.70% (113/135)	93.10% (27/29)	100.00% (26/26)
Total	83.06% (103/124)	83.70% (113/135)	93.10% (27/29)	100.00% (26/26)

Changelog

- 2026-07-07 - Initial report
- 2026-07-08 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over $500 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, Polymarket, PancakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland

- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.